

QRコード読み取り時の文字列文字化けについて



<https://github.com/akutor...>

こんなもの作ってました

保守連絡時などで業務ID?(おそらく商品種別的なもの?)をバーコードにして、スキャナーで読み取り→入力画面に反映させる人が社内にはいた。

それに「感動」して、個人的にQRコード・バーコード生成ツールを作って個人的に運用していました。

内容の割にはちょっとは役に立っている模様

~~最近AIを使ってツールを作るのは楽しいと感じる!!!!~~

何があったのか



QR・バーコード生成ツールを自作して運用していたところ、業務用スキャナー（Inateck BCST-72）で**日本語入りのQRコードを読み取ると文字化けする**、という問題が発生(windows11で発生)

スマホのカメラで読む分には問題なし。業務スキャナー読み取りで文字化けしていた

PCの設定などで使おうと思っていたが、出力が文字化けして使い物にならないため、業務用スキャナーでの文字化けを回避する方法を調査することにした

最初の問題：文字コード

QRコードの仕様上、中身はただのバイト列。「どの文字コードで解釈するか」の指定をもたない。(ただ、特に指定がなければISO/IEC 8859-1で解釈するのが仕様上のデフォルト)

一方漢字・かなについては、通称「Shift-JIS」で解釈される仕様の模様

自分が使っていたIMEがGoogle IMEだったため、Shift-JISでエンコードすれば日本語の文字化けが解消された

→ 対策として UTF-8 / Shift-JIS を選んで生成できる機能を実装した

ただ、Microsoft IMEを使っていると、Shift-JISでエンコードしても文字化けした(BOM付きUTF-8でエンコードしても文字化けした)

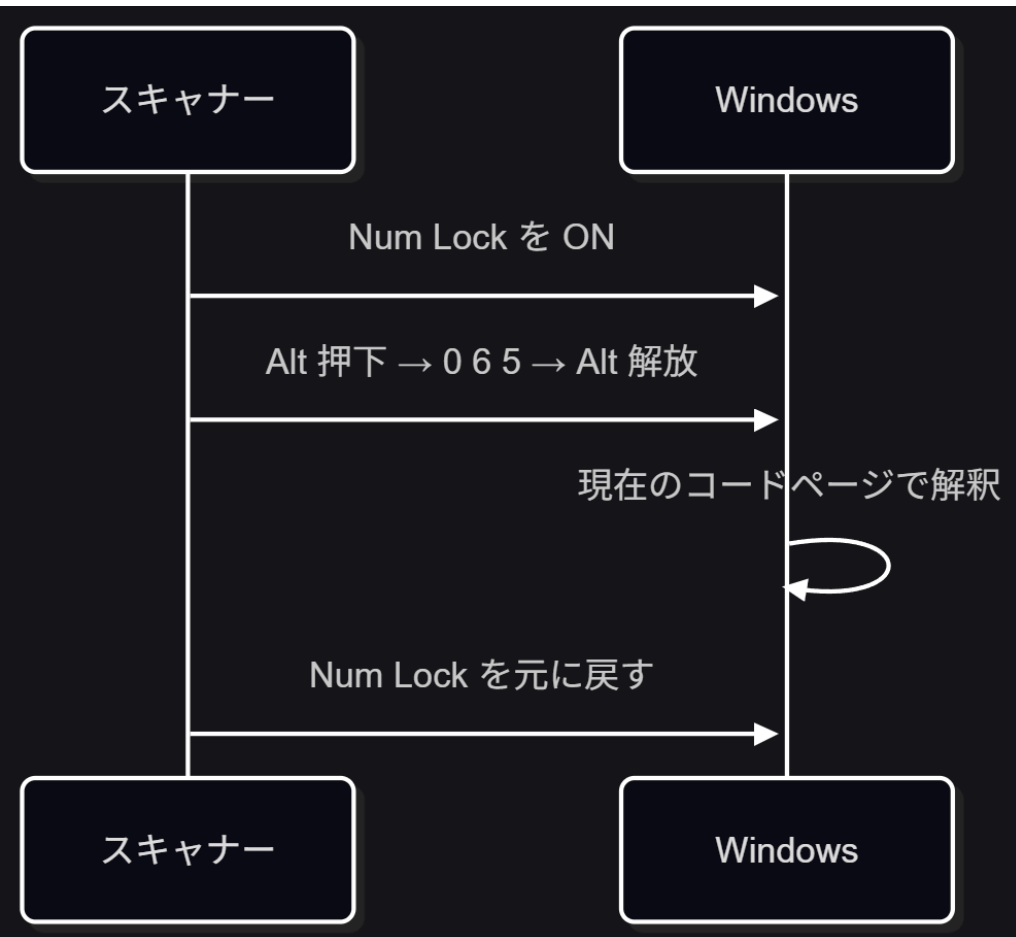
Microsoft IMEの解釈文字コード(?)は UTF-16 のため、どうあがいても文字化けした

Windows アプリケーションは Unicode の UTF-16 実装を使用します。UTF-16 では、ほとんどの文字は 2 バイト コードで識別されます。

実際にUTF-16エンコードでQRコード自体は問題なく生成できた。生成したQRコードは変えずに読み取り側の条件だけを変えて検証したところ、PC側のIME（Microsoft IME / Google IME）×Inateckスキャナー側の解釈文字コード設定（UTF-8 / Shift-JIS / そのまま）の全パターンで正しく読み取れなかった

そもそもUTF-16エンコードのQRコード自体が一般的ではないため、別のアプローチを取る必要があった

余談：Num Lockが点滅する



検証中、スキャンするたびに Num Lock が一瞬点滅すること気づいた

調べると "Keypad Emulation" という仕組みで、1文字ごとに Alt + テンキー数字で文字コードを直接注入している

Alt + テンキーはOEM/ANSIコードページ内の文字を入力する機能で、任意のUnicodeを直接扱えるわけではない模様

ロケール設定やコードページに依存するようなものなのでこれも原因の一つだったかも

そもそもなぜこのような「入力に関する問題が起きていたか」というと、スキャナーが **キーボードエミュレーション** で文字列を注入していたから

スキャナー(これに限らずOCRスキャナーなども含む)は、PCに接続すると仮想的なキーボードとして認識される。

スキャンした文字列を、仮想的なキーボードから直接注入することで、アプリ側は「ユーザーがキーボードで入力した」と同じ扱いになる

そのためIMEやキーボード配列の設定をダイレクトに受けやすい(その分設定などはしなくてよいという利点もあるけど・・・)

これを完全に回避するためには「キーボードとしての入力」とは別のアプローチを取る必要があった

スキャナーの接続方式

スキャナーはおおよそ3種類程度の接続方式を提供している

- **HID キーボードエミュレーション**

- ゼロ設定で動くのが売り。ただし非ASCIIはAlt+Numpad等の力技に頼る

- **COM ポート（仮想シリアル）モード**

- USB や Bluetooth Classic の SPP（Serial Port Profile）で「ただのシリアルポート」として見える
- 生バイト列をアプリ側が自分でデコードするので IME もキーボードエミュレーションも一切関係ない
 - Zebra/Honeywell/Datalogic 等の老舗スキャナーは数十年前からこれに対応しており、業務システムでは実はこちらが「本命」の接続方式

- **BLE GATT + 独自 SDK**

- Bluetooth Classic の SPP に相当する標準規格が BLE には存在しないため、各社が独自プロトコル を作らざるを得ない
- Inateck が特別面倒なのではなく、BLE 専用機はこの面倒さから逃れられないというのが実情

GATTモード

GATTとは

GATT (Generic attribute profile) ー汎用アトリビュートプロファイルは、ATT (アトリビュートプロトコル) を用いてデータを構造化する方法と、アプリケーション間でのやり取りの方法を定義します。

Bluetooth low energyのアプリケーションは、すべてこのGATTを使用して構築されることから、GATTはBluetooth low energyのデータ転送の主軸となるものです。

実際に Bluetooth LE Explorer (割と激古) で接続し Service / Characteristic を探索。
0xAE00 サービス配下に怪しいカスタムキャラクタースティック 0xAE01 / 0xAE02 を
発見した (Read Not Permitted = Notify専用の可能性)。

Device Services Page

Discover and Pair

Virtual Peripheral

Virtual Keyboard

Advertisement Monitor

Advertisement Beacon

Settings

BT Address: ab:2b:00:26:3a:4a

Number of Services: 8

Number of service changed events: 0

Number of Advertisement Services: 1

BT 4.2 Secure Connection: False

Device Connected: True

Refresh

Start Transaction

Service Name: **GenericAccess**

Service UUID: 00001800-0000-1000-8000-00805f9b34fb

Characteristic Name: **DeviceName** - Characteristic Short UUID: **0x2A00** - User Description: - Handle: **49** - **0x00000031** - Value: **NULL**

Characteristic Name: **Appearance** - Characteristic Short UUID: **0x2A01** - User Description: - Handle: **51** - **0x00000033** - Value: **NULL**

Characteristic Name: **PeripheralPreferredConnectionParameters** - Characteristic Short UUID: **0x2A04** - User Description: - Handle: **53** - **0x00000035** - Value: **NULL**

Service Name: **GenericAttribute**

Service UUID: 00001801-0000-1000-8000-00805f9b34fb

Service Name: **DeviceInformation**

Service UUID: 0000180a-0000-1000-8000-00805f9b34fb

Characteristic Name: **ManufacturerNameString** - Characteristic Short UUID: **0x2A29** - User Description: - Handle: **57** - **0x00000039** - Value: **6C-69-78-69-6E-40-53-54-2D-37-32-41-49-00-4A-3A-26-00-2B-AB**

Service Name: **Battery**

Service UUID: 0000180f-0000-1000-8000-00805f9b34fb

Characteristic Name: **BatteryLevel** - Characteristic Short UUID: **0x2A19** - User Description: - Handle: **60** - **0x0000003C** - Value: **59**

Service Name: **6384**

Service UUID: 000018f0-0000-1000-8000-00805f9b34fb

Characteristic Name: **10992** - Characteristic Short UUID: **0x2AF0** - User Description: - Handle: **64** - **0x00000040** - Value: **Read Not Permitted**

Service Name: **65258**

Service UUID: 0000feea-0000-1000-8000-00805f9b34fb

Characteristic Name: **MagneticFluxDensity3D** - Characteristic Short UUID: **0x2AA1** - User Description: - Handle: **68** - **0x00000044** - Value: **Read Not Permitted**

Service Name: **44544**

Service UUID: 0000ae00-0000-1000-8000-00805f9b34fb

Characteristic Name: **44545** - Characteristic Short UUID: **0xAE01** - User Description: - Handle: **129** - **0x00000081** - Value: **Read Not Permitted**

Characteristic Name: **44546** - Characteristic Short UUID: **0xAE02** - User Description: - Handle: **131** - **0x00000083** - Value: **Read Not Permitted**

じゃあどうすればいいのか

このスキャナーのキーボードエミュレーションは、先述のAlt+テンキーによるコードページ直接注入で文字を送っている

かな漢字変換のようなIME**本来の変換ロジック**は関係しないはずだが、Shift-JISで生成した場合、実際にはIMEの種類（Google IME / Microsoft IME）によって結果が変わった。

原因は特定できていない。

じゃあ業務システムでスキャナーを使うならどうするか

- **選べるなら**：COM/SPP対応のスキャナーを選ぶ。生バイト列を自分でデコードできるので、この手の文字化けはそもそも起きない
- **選べない場合**：QRにはIDだけ入れて、日本語は**DB側**で引く設計にする
 - JAN/ISBNのような数字のみの業務コードなら、そもそもこの手の文字化け自体が起きにくい

まとめ

- QRコード自体は終始無罪だった
- 本当の犯人は「キーボードエミュレーション」という転送方式
- UTF-16は生成はできるが実用性ゼロ
- GATTモードも試したが、独自（ベンダー固有）プロファイルの壁にあたった
- 業務システムなら、選べるなら **COM/SPP対応** のスキャナーを選ぶのが一番